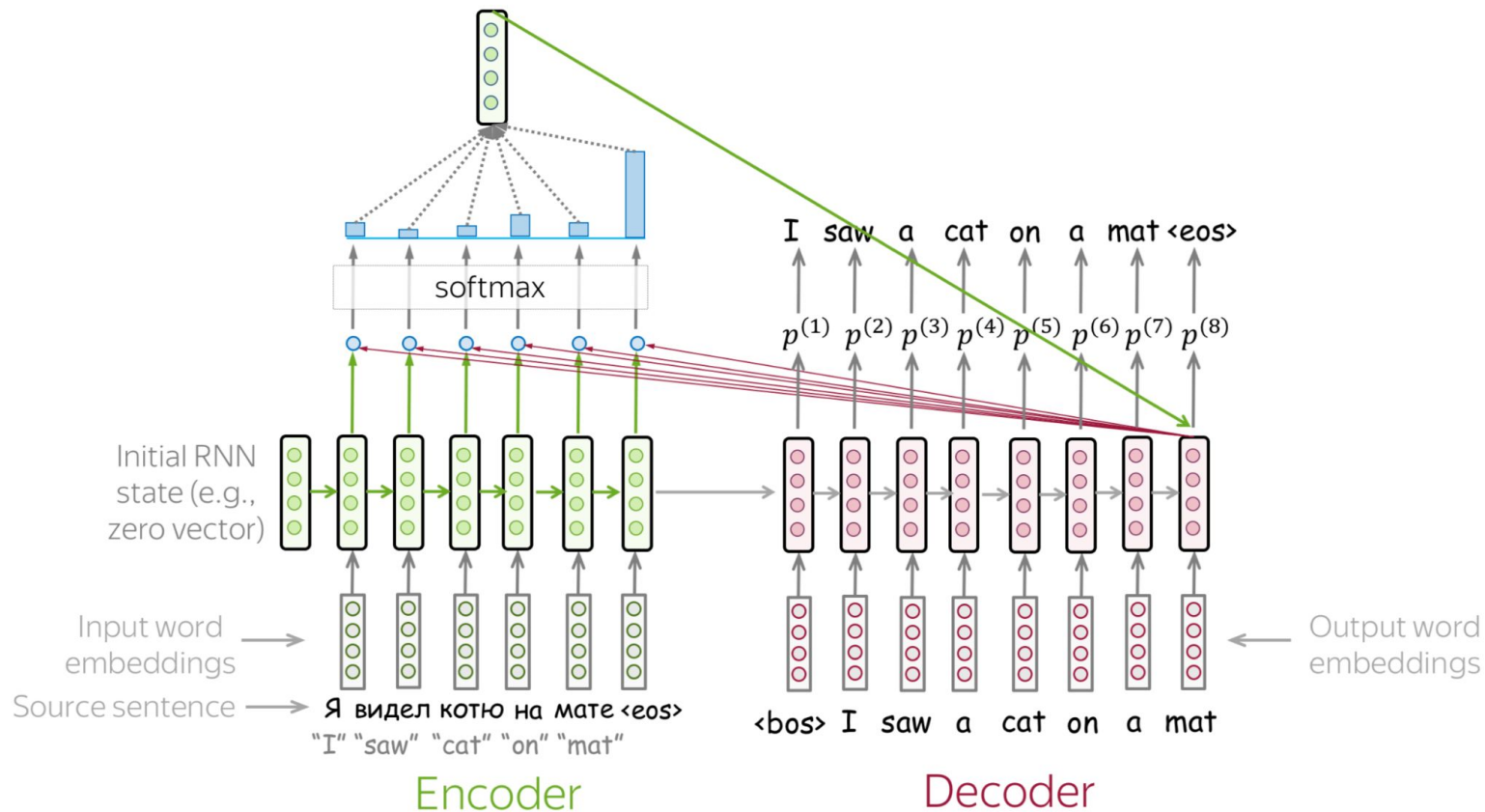
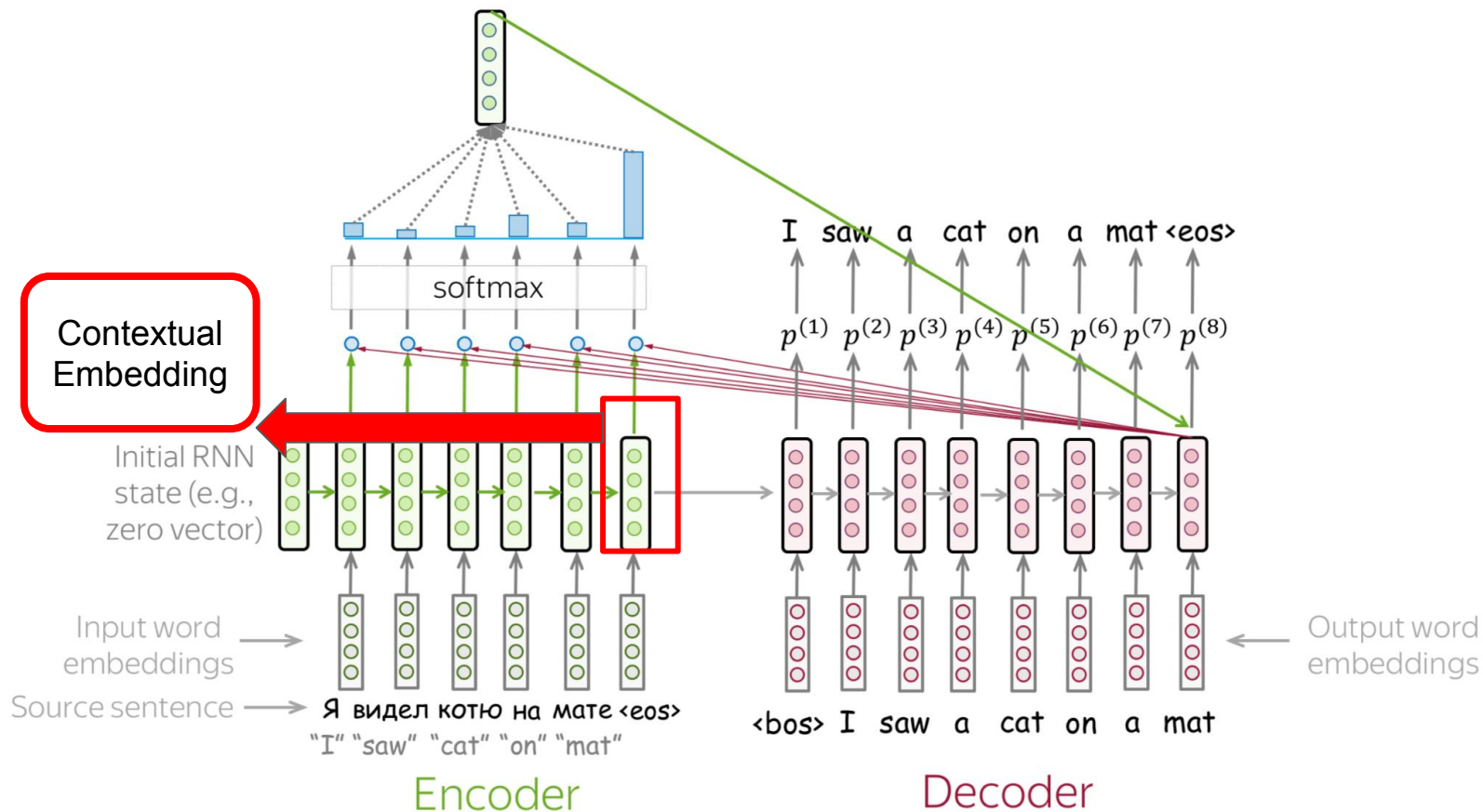
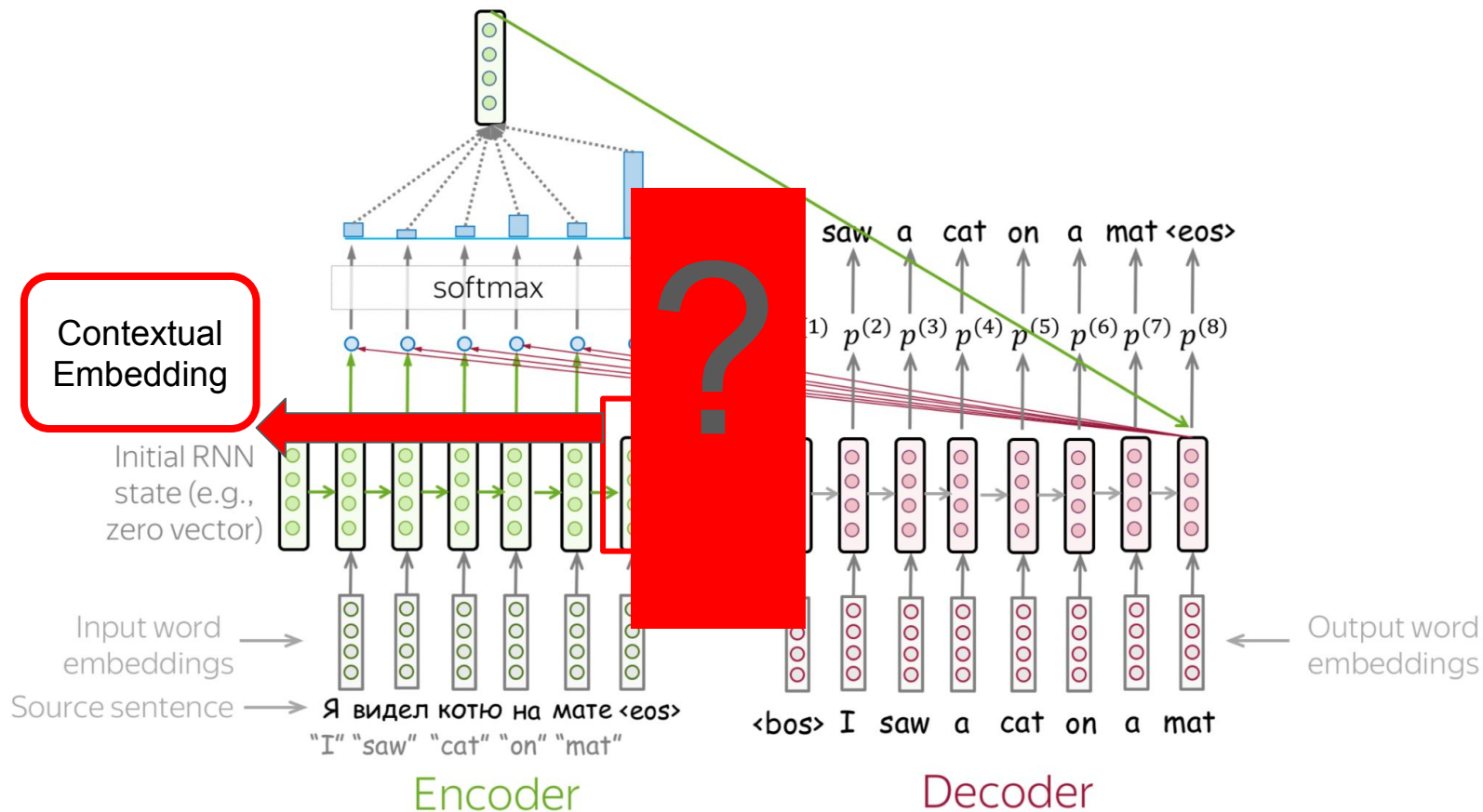


Recap : Neural Machine Translation - LSTM-Attention Based



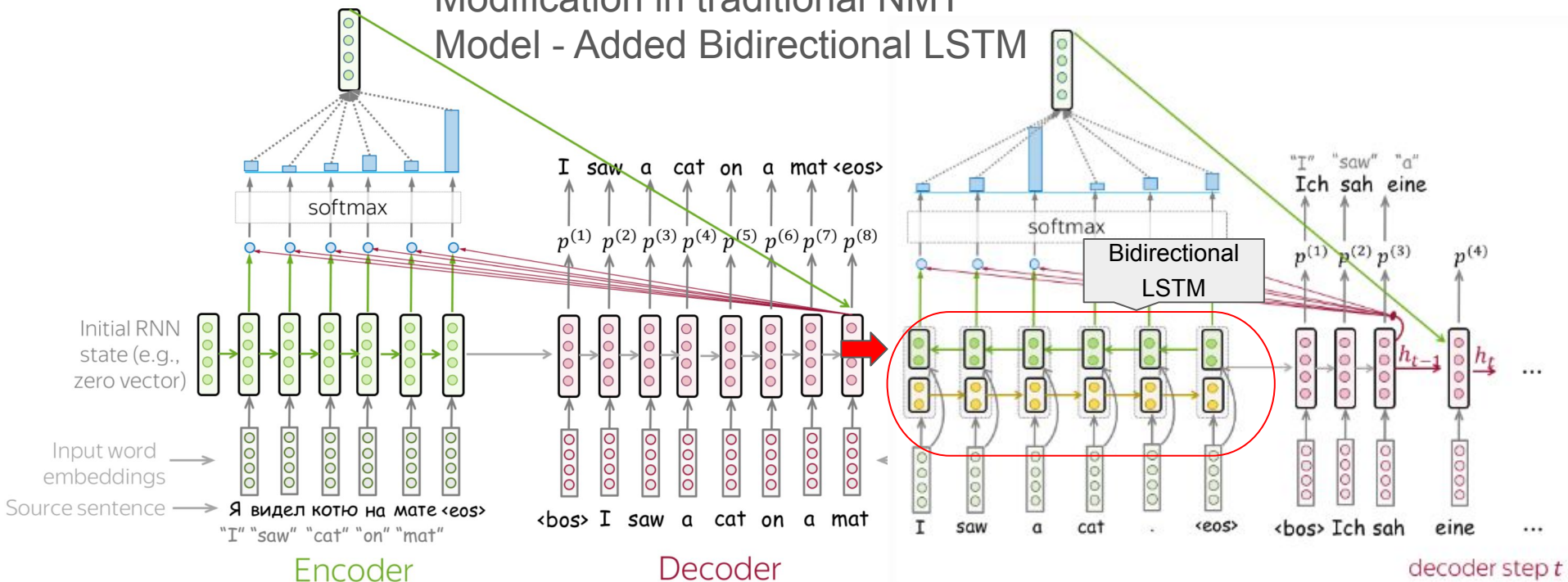


Contextual Embedding - Any Improvement ?



CoVe : Contextualized Word Vectors Learned in Translation

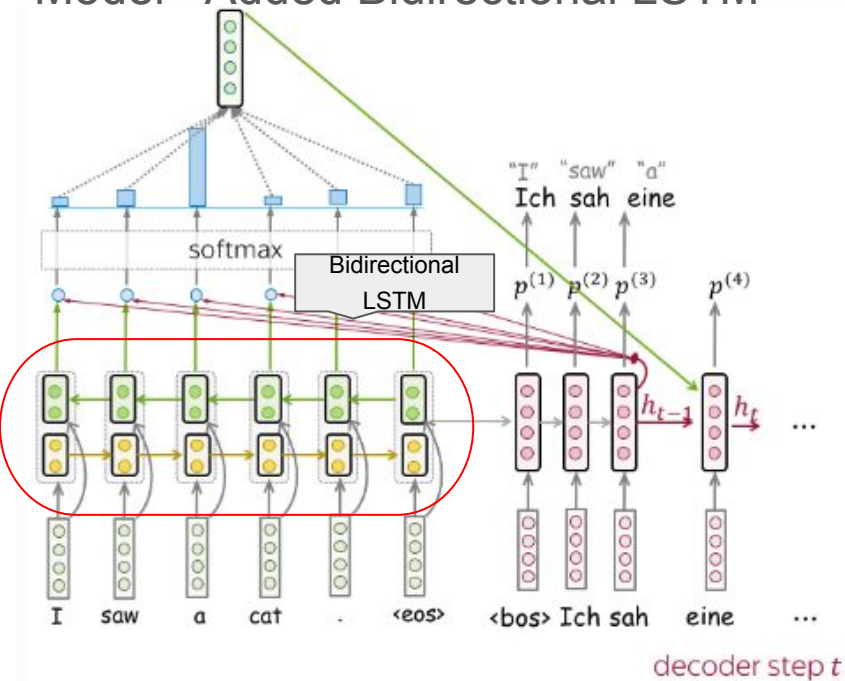
Modification in traditional NMT Model - Added Bidirectional LSTM



CoVe : Contextualized Word Vectors Learned in Translation

Modification in traditional NMT Model - Added Bidirectional LSTM

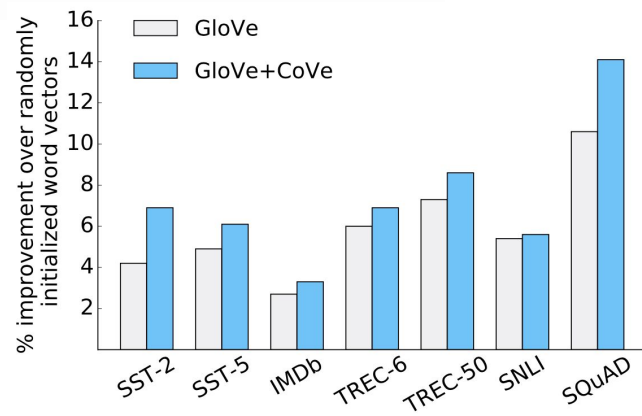
+ Concatenate with GloVe Embedding



CoVe
GloVe

Concatenate GloVe
and CoVe vectors
for each token

This contextual
Embedding Improved
the result of
Downstream Tasks



CoVe Improved the the Contextual Embedding. But anything else ? We can Improve?

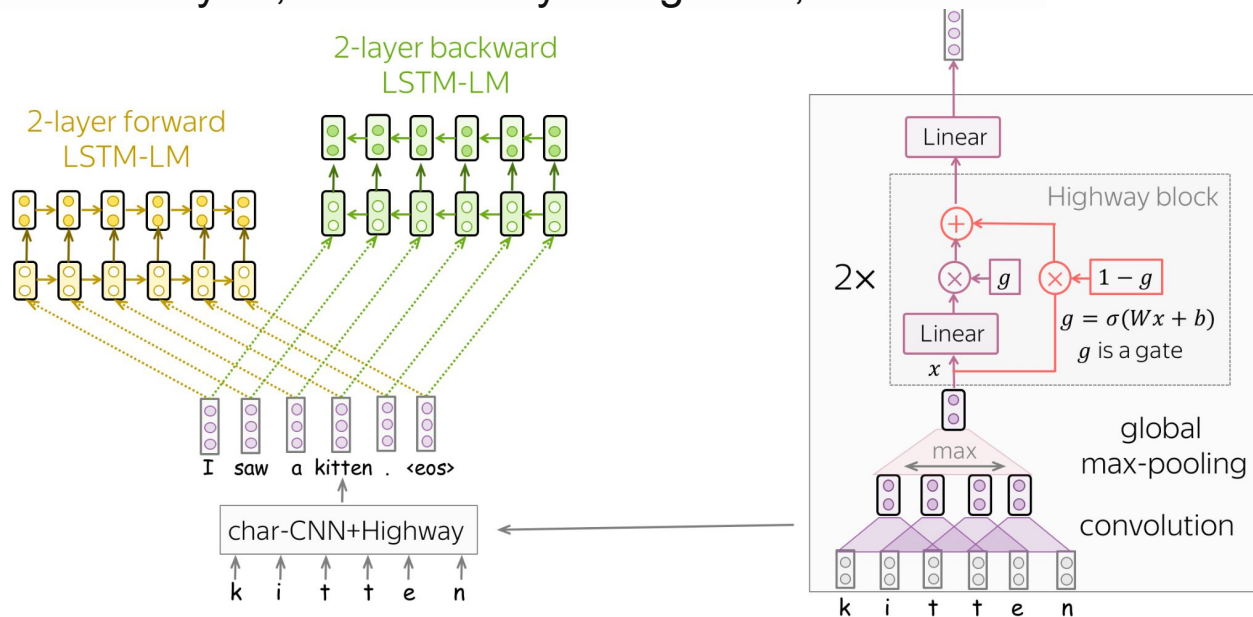
Yes, By using ELMo!

Contextual Embedding - Embeddings from Language Models (ELMo)

Differently from CoVe, ELMo uses representations not from NMT model, but from a **language model**. Just by **replacing** word embeddings (GloVe) with **embeddings from LM** they got a huge **improvement** for several tasks such as question answering, coreference resolution, sentiment analysis, named entity recognition, and others.

Modification for ELMo

Forward and Backward
LSTM-LMs
on top of char-CNN

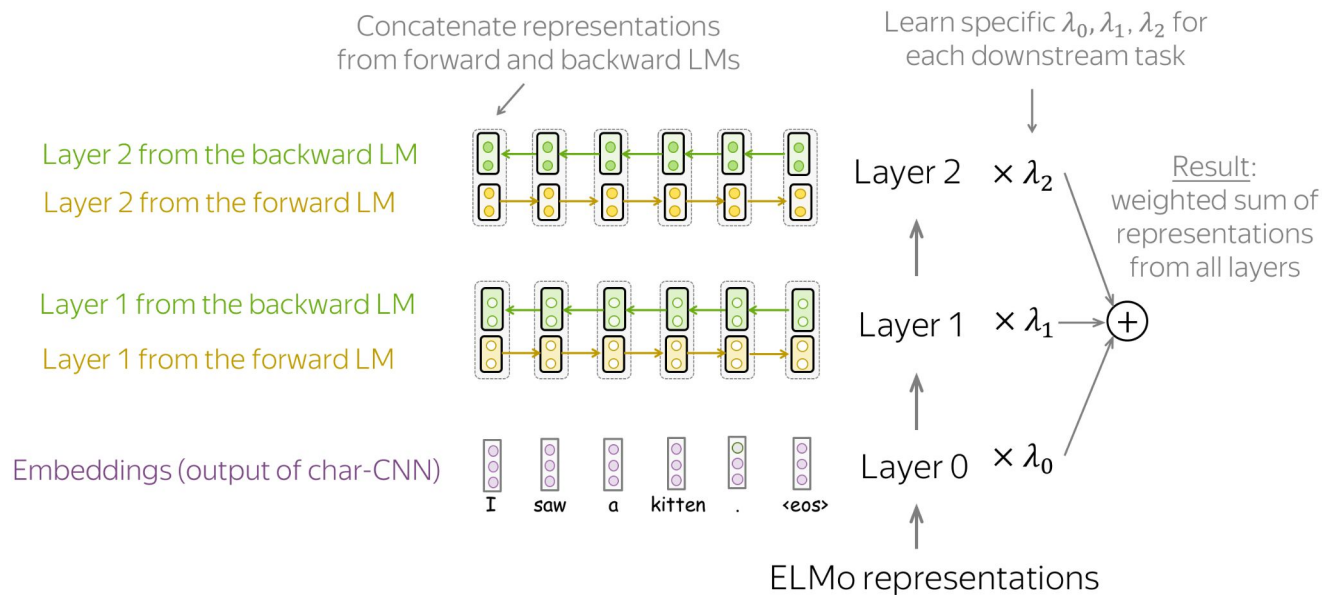


Contextual Embedding - Embeddings from Language Models (ELMo)

Modification for ELMo

Overall, ELMo representations have three layers:

- layer 0 (embeddings) - output of the character-level CNN;
- layer 1 - concatenated representations from layer 1 of both forward and backward LSTMs;
- layer 2 - concatenated representations from layer 2 of both forward and backward LSTMs.



Contextual Embedding - Embeddings from Language Models (ELMo)

Task	Previous SOTA		ELMo + Baseline
SQuAD	SAN	84.4	85.8
SNLI	Chen et al (2017)	88.6	88.7 +/- 0.17
SRL	He et al (2017)	81.7	84.6
Coref	Lee et al (2017)	67.2	70.4
NER	Peters et al (2017)	91.93 +/- 0.19	92.22 +/- 0.10
Sentiment (5-class)	McCann et al (2017)	53.7	54.7 +/- 0.5

ELMo improve the contextual embedding representation!!



Transformers

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

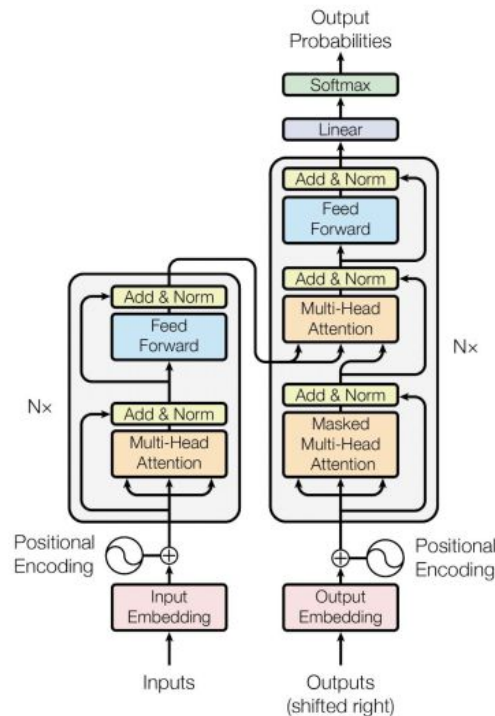
Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukasz.kaiser@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions



Transformers

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions

- Scale efficiently
- Parallel process
- Attention to input meaning

Transformer

Transformer

I arrived at the **bank** after crossing thestreet? ...river?
What does **bank** mean in this sentence?



I've no idea: let's wait
until I read the end

RNNs

$O(N)$ steps to process a
sentence with length N



I don't need to wait - I
see all words at once!

Transformer

Constant number of steps
to process any sentence

$$s_4 = h_1^D * h_4^E \quad a_4 = \sigma(s_4)$$

$$s_3 = h_1^D * h_3^E \quad a_3 = \sigma(s_3)$$

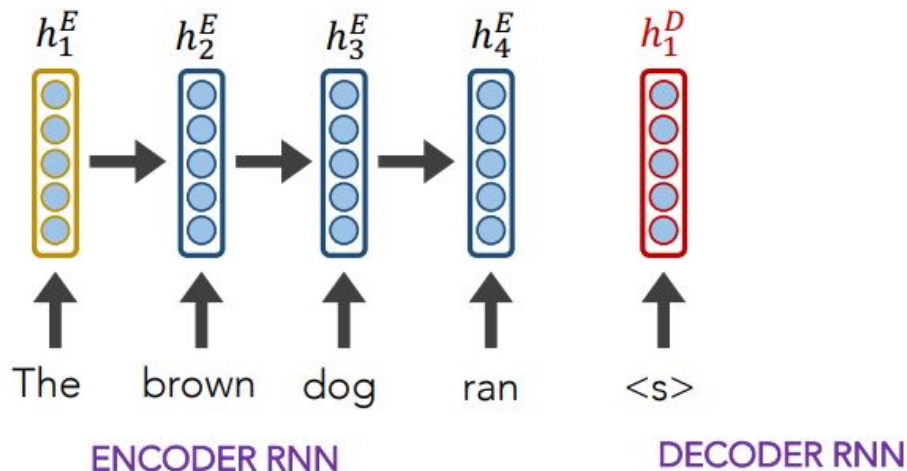
$$s_2 = h_1^D * h_2^E \quad a_2 = \sigma(s_2)$$

$$s_1 = h_1^D * h_1^E \quad a_1 = \sigma(s_1)$$

We multiply each encoder's hidden layer by its a_i^1 attention weights to create a context vector c_1^D

$$c_1^D = a_1 \cdot h_1^E + a_2 \cdot h_2^E + a_3 \cdot h_3^E + a_4 \cdot h_4^E$$

Attention



$$s_4 = q_2 \cdot k_4 \quad a_4 = \sigma(s_4/8)$$

$$s_3 = q_2 \cdot k_3 \quad a_3 = \sigma(s_3/8)$$

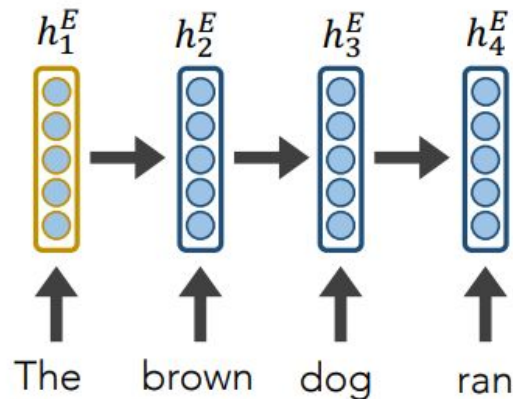
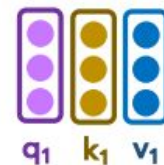
$$s_2 = q_2 \cdot k_2 \quad a_2 = \sigma(s_2/8)$$

$$s_1 = q_2 \cdot k_1 \quad a_1 = \sigma(s_1/8)$$

We multiply each word's **value** vector by its a_i^1 **attention weights** to **create** a better vector z_1

$$z_1 = a_1 \cdot v_1^E + a_2 \cdot v_2^E + a_3 \cdot v_3^E + a_4 \cdot v_4^E$$

Self-Attention



ENCODER RNN

Query, Key, and Value in Self-Attention

Formally, this intuition is implemented with a query-key-value attention. Each input token in self-attention receives three representations corresponding to the roles it can play:

- query - asking for information;
- key - saying that it has some information;
- value - giving the information.

The query is used when a token looks at others - it's seeking the information to understand itself better. The key is responding to a query's request: it is used to compute attention weights. The value is used to compute attention output: it gives information to the tokens which "say" they need it (i.e. assigned large weights to this token).

Self Attention

Each vector receives three representations ("roles")

$$\begin{bmatrix} W_Q \end{bmatrix} \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{blue} \\ \text{blue} \\ \text{blue} \end{bmatrix}$$

Query: vector **from** which the attention is looking

"Hey there, do you have this information?"

$$\begin{bmatrix} W_K \end{bmatrix} \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{yellow} \\ \text{yellow} \\ \text{yellow} \end{bmatrix}$$

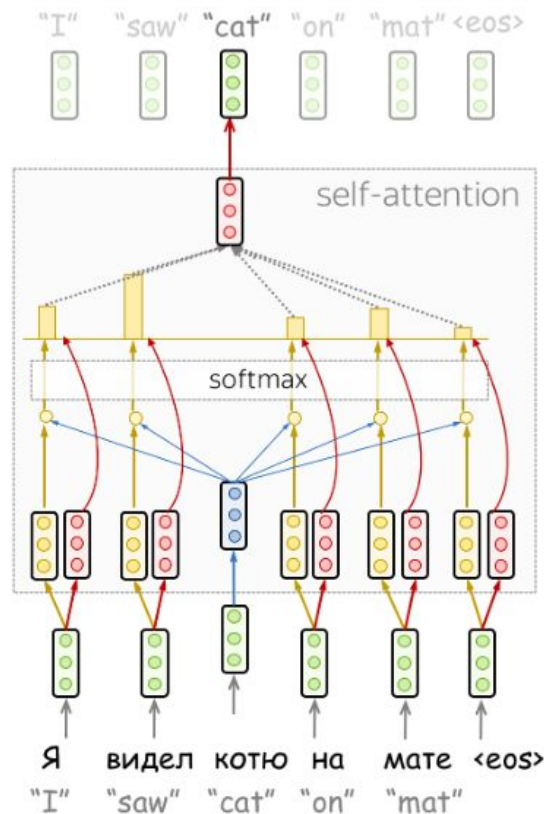
Key: vector **at** which the query looks to compute weights

"Hi, I have this information – give me a large weight!"

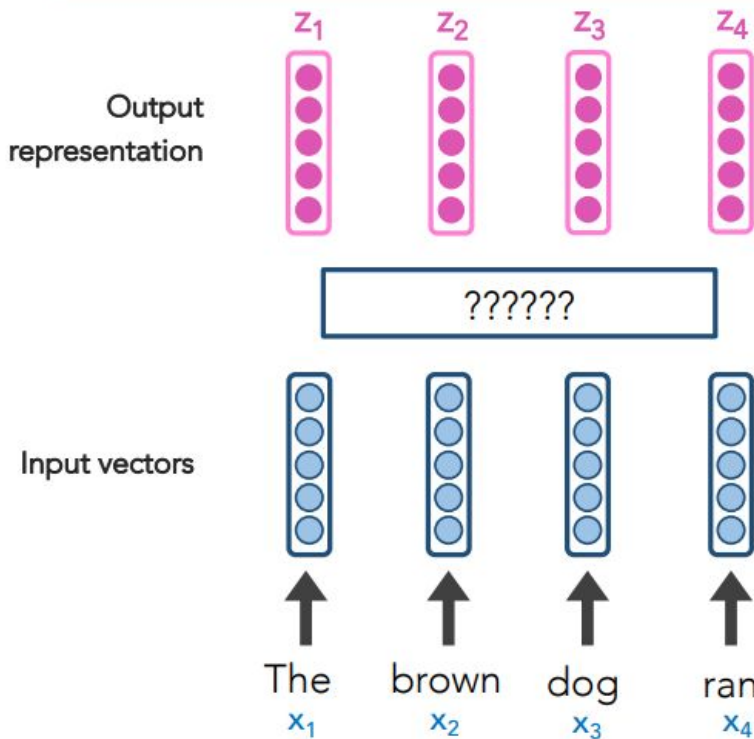
$$\begin{bmatrix} W_V \end{bmatrix} \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{red} \\ \text{red} \\ \text{red} \end{bmatrix}$$

Value: their weighted sum is attention output

"Here's the information I have!"

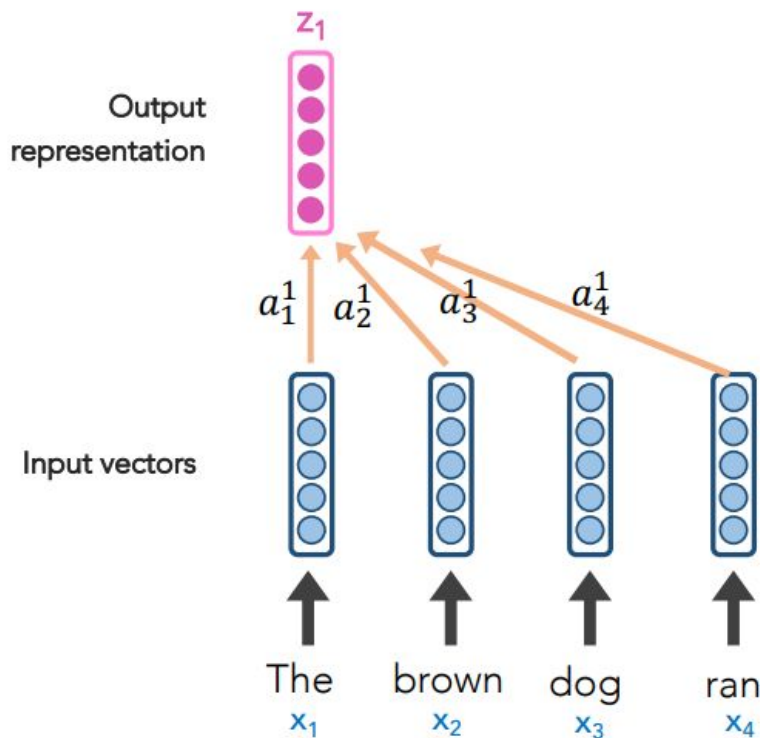


Self-Attention



Self-Attention's goal is to create great representations, z_i , of the input

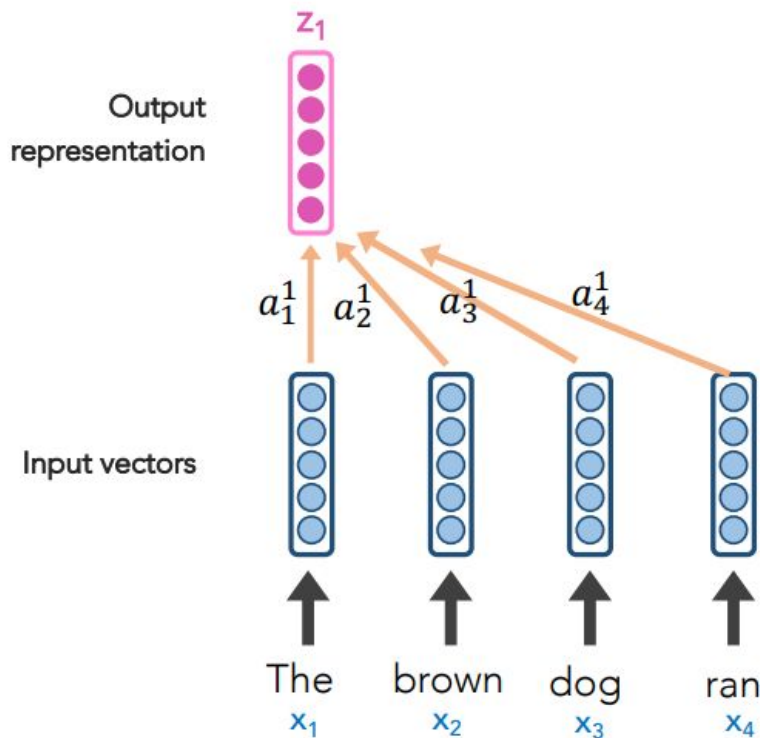
Self-Attention



Self-Attention's goal is to create great representations, z_i , of the input

z_1 will be based on a weighted contribution of x_1, x_2, x_3, x_4

Self-Attention

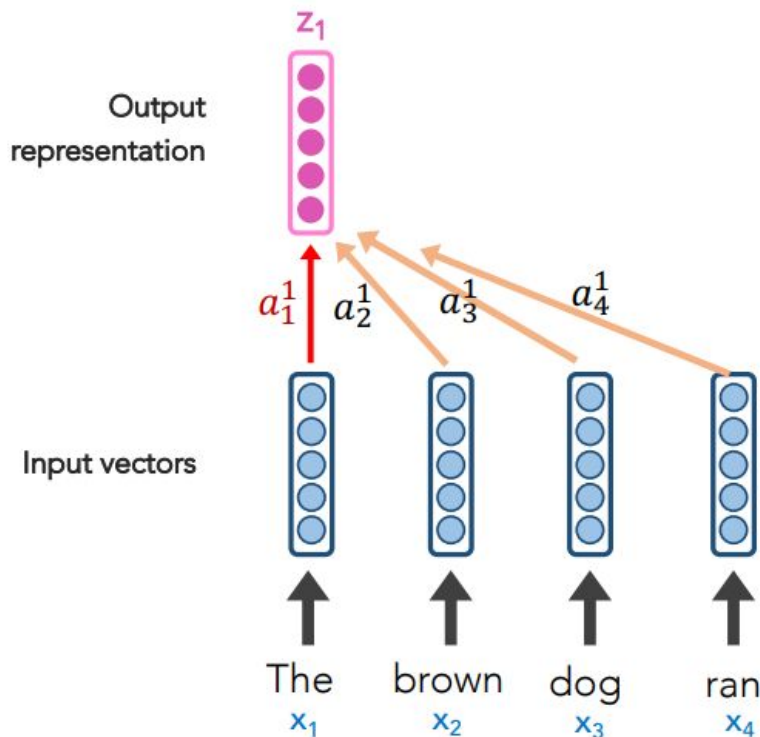


Self-Attention's goal is to create great representations, z_i , of the input

z_1 will be based on a weighted contribution of x_1, x_2, x_3, x_4

a_i^1 is "just" a weight. More is happening under the hood, but it's effectively weighting versions of x_1, x_2, x_3, x_4

Self-Attention



Under the hood, each x_i has 3 small, associated vectors.

For example, x_1 has:

- Query q_i
- Key k_i
- Value v_i

Self-Attention

Step 1: Our Self-Attention Head I has just 3 weight matrices W_q , W_k , W_v in total. These same 3 weight matrices are multiplied by each x_i to create all vectors:

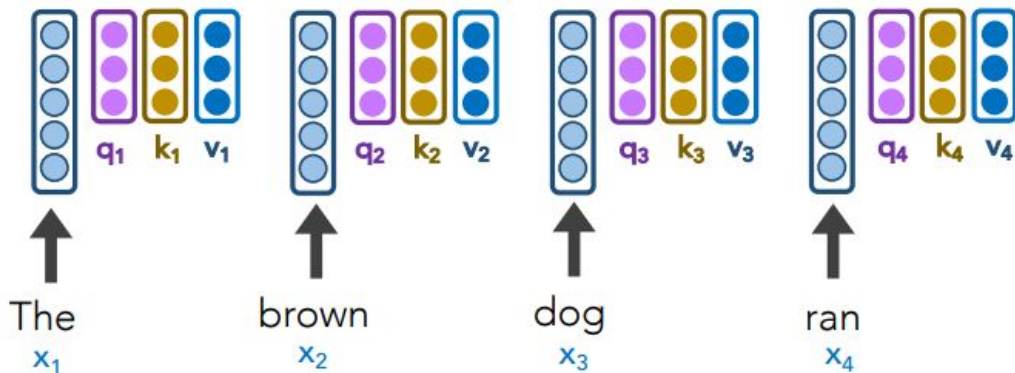
$$q_i = w_q x_i$$

$$k_i = w_k x_i$$

$$v_i = w_v x_i$$

Under the hood, each x_i has 3 small, associated vectors. For example, x_1 has:

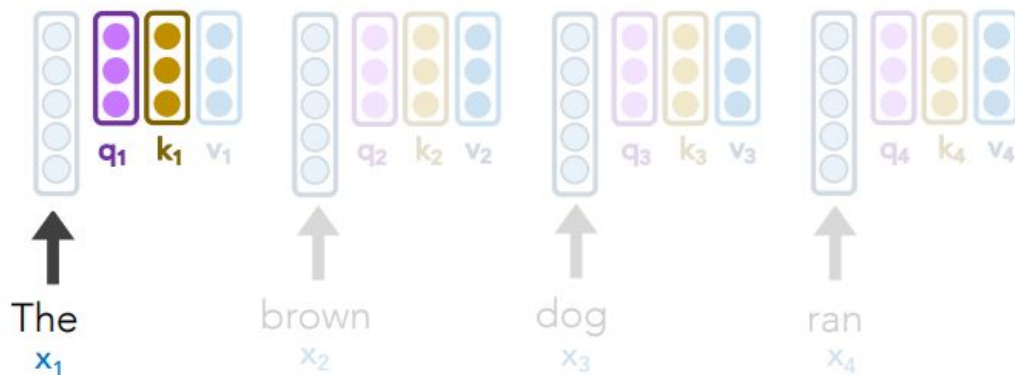
- Query q_1
- Key k_1
- Value v_1



Self-Attention

Step 2: For word x_1 , let's calculate the scores s_1, s_2, s_3, s_4 , which represent how much attention to pay to each respective "word" v_i

$$s_1 = q_1 \cdot k_1 = 112$$

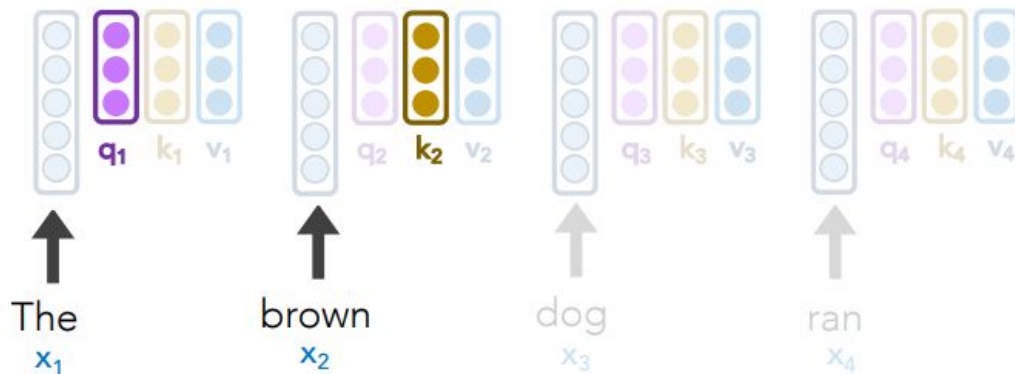


Self-Attention

Step 2: For word x_1 , let's calculate the scores s_1, s_2, s_3, s_4 , which represent how much attention to pay to each respective "word" v_i

$$s_2 = q_1 \cdot k_2 = 96$$

$$s_1 = q_1 \cdot k_1 = 112$$



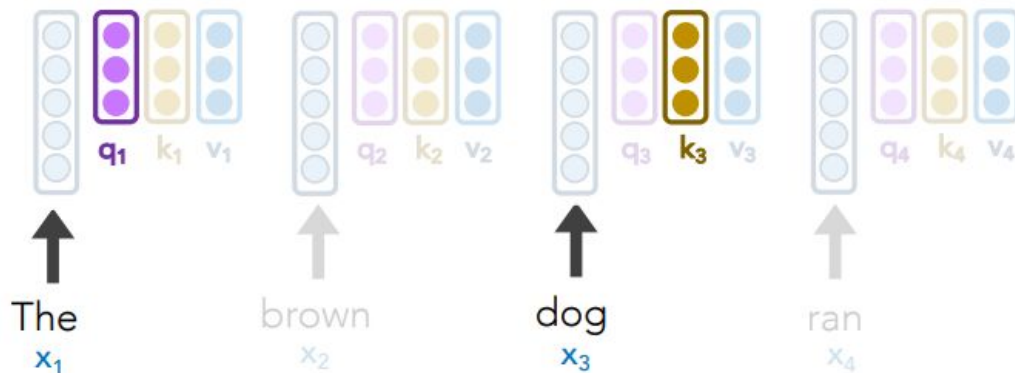
Self-Attention

Step 2: For word x_1 , let's calculate the scores s_1, s_2, s_3, s_4 , which represent how much attention to pay to each respective "word" v_i

$$s_3 = q_1 \cdot k_3 = 16$$

$$s_2 = q_1 \cdot k_2 = 96$$

$$s_1 = q_1 \cdot k_1 = 112$$



Self-Attention

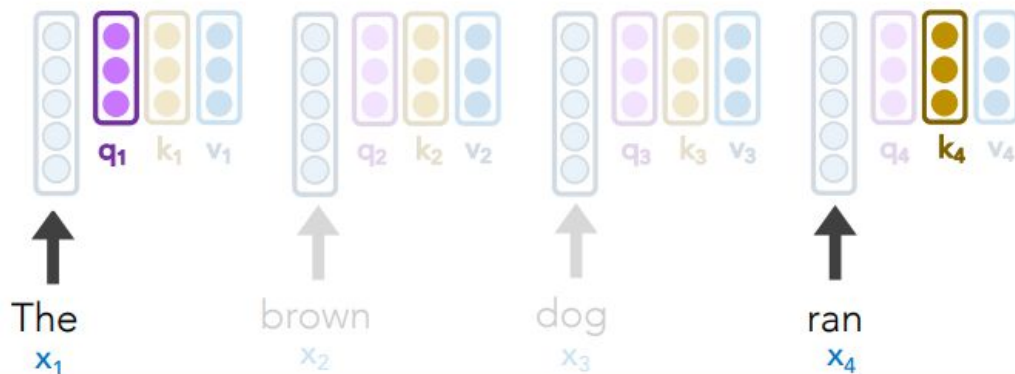
Step 2: For word x_1 , let's calculate the scores s_1, s_2, s_3, s_4 , which represent how much attention to pay to each respective "word" v_i

$$s_4 = q_1 \cdot k_4 = 8$$

$$s_3 = q_1 \cdot k_3 = 16$$

$$s_2 = q_1 \cdot k_2 = 96$$

$$s_1 = q_1 \cdot k_1 = 112$$



Self-Attention

Step 3: Our scores s_1, s_2, s_3, s_4 don't sum to 1. Let's divide by $\sqrt{\text{len}(k_i)}$ and **softmax** it

$$s_4 = q_1 \cdot k_4 = 8$$

$$a_4 = \sigma(s_4/8) = 0$$

$$s_3 = q_1 \cdot k_3 = 16$$

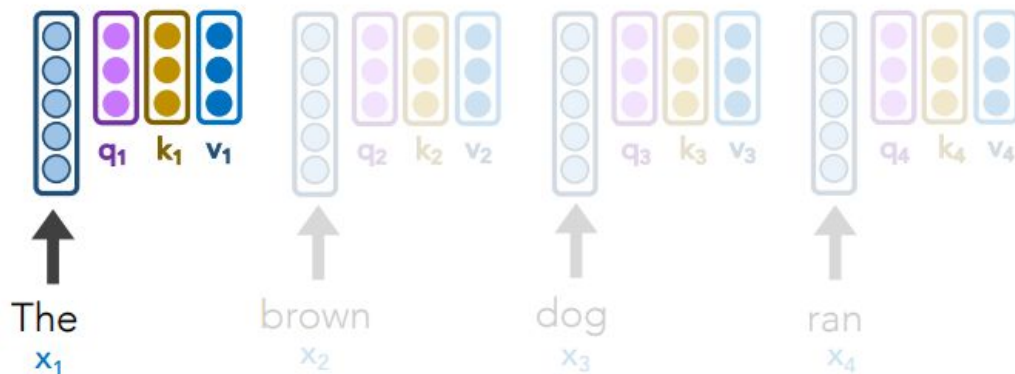
$$a_3 = \sigma(s_3/8) = .01$$

$$s_2 = q_1 \cdot k_2 = 96$$

$$a_2 = \sigma(s_2/8) = .12$$

$$s_1 = q_1 \cdot k_1 = 112$$

$$a_1 = \sigma(s_1/8) = .87$$



Self-Attention

Step 3: Our scores s_1, s_2, s_3, s_4 don't sum to 1. Let's divide by $\sqrt{\text{len}(k_i)}$ and **softmax** it

$$s_4 = q_1 \cdot k_4 = 8$$

$$a_4 = \sigma(s_4/8) = 0$$

$$s_3 = q_1 \cdot k_3 = 16$$

$$a_3 = \sigma(s_3/8) = .01$$

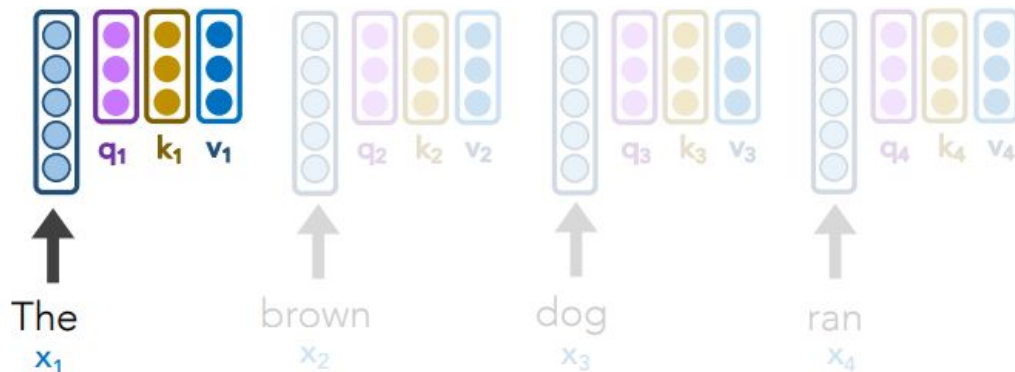
$$s_2 = q_1 \cdot k_2 = 96$$

$$a_2 = \sigma(s_2/8) = .12$$

$$s_1 = q_1 \cdot k_1 = 112$$

$$a_1 = \sigma(s_1/8) = .87$$

Instead of these a_i values directly weighting our original x_i word vectors, they directly weight our v_i vectors.



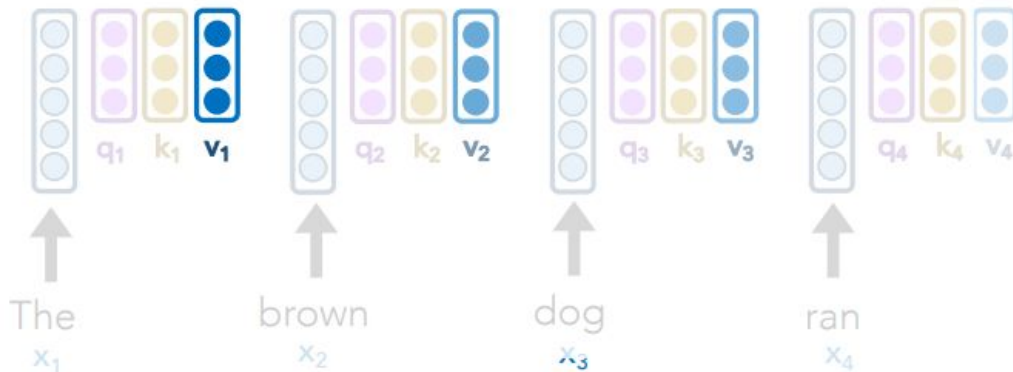
Self-Attention

Step 4: Let's weight our $\mathbf{v}_i$ vectors and simply sum them up!

z_1



$$\begin{aligned} z_1 &= a_1 \cdot \mathbf{v}_1 + a_2 \cdot \mathbf{v}_2 + a_3 \cdot \mathbf{v}_3 + a_4 \cdot \mathbf{v}_4 \\ &= 0.87 \cdot \mathbf{v}_1 + 0.12 \cdot \mathbf{v}_2 + 0.01 \cdot \mathbf{v}_3 + 0 \cdot \mathbf{v}_4 \end{aligned}$$



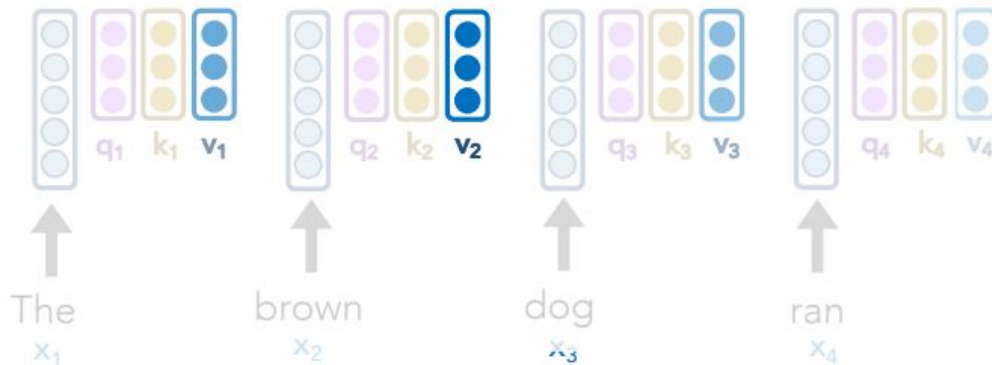
Self-Attention

Step 5: We repeat this for all other words, yielding us with great, new z_i representations!

z_2

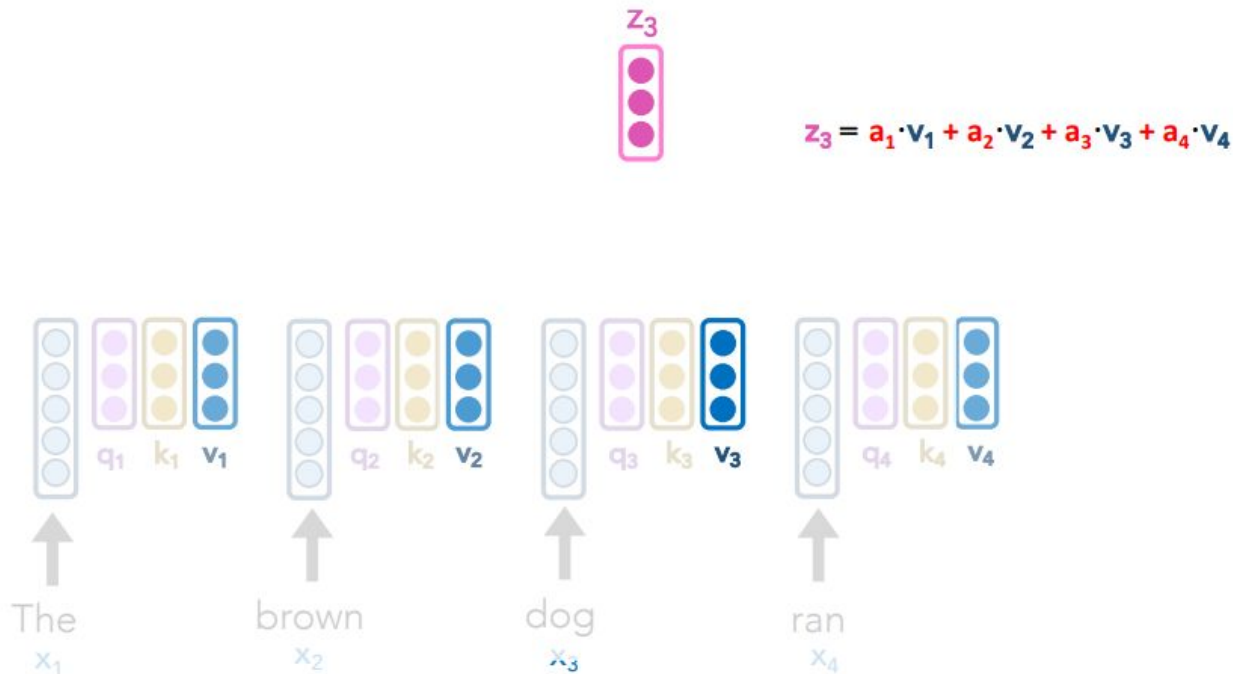


$$z_2 = a_1 \cdot v_1 + a_2 \cdot v_2 + a_3 \cdot v_3 + a_4 \cdot v_4$$



Self-Attention

Step 5: We repeat this for all other words, yielding us with great, new z_i representations!

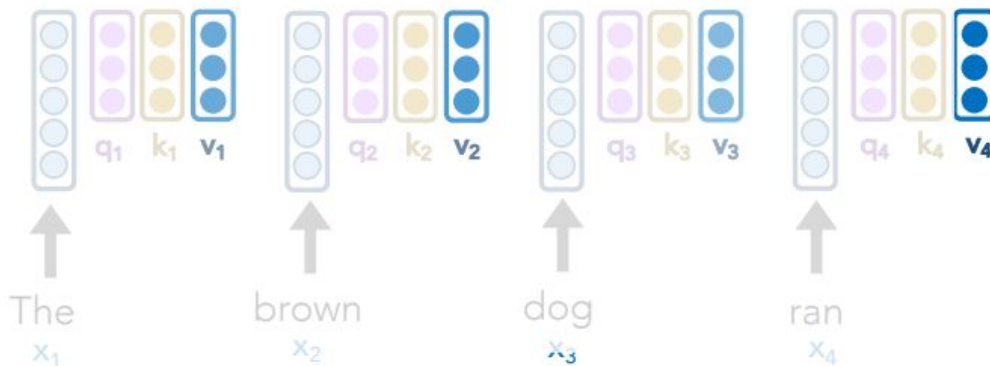


Self-Attention

Step 5: We repeat this for all other words, yielding us with great, new z_i representations!

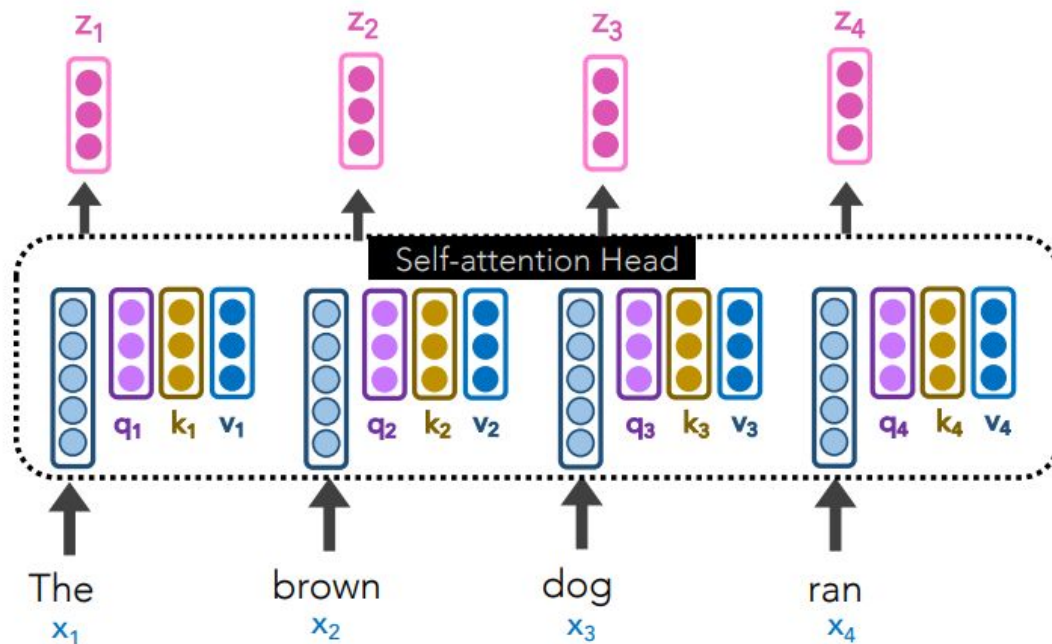


$$z_4 = a_1 \cdot v_1 + a_2 \cdot v_2 + a_3 \cdot v_3 + a_4 \cdot v_4$$

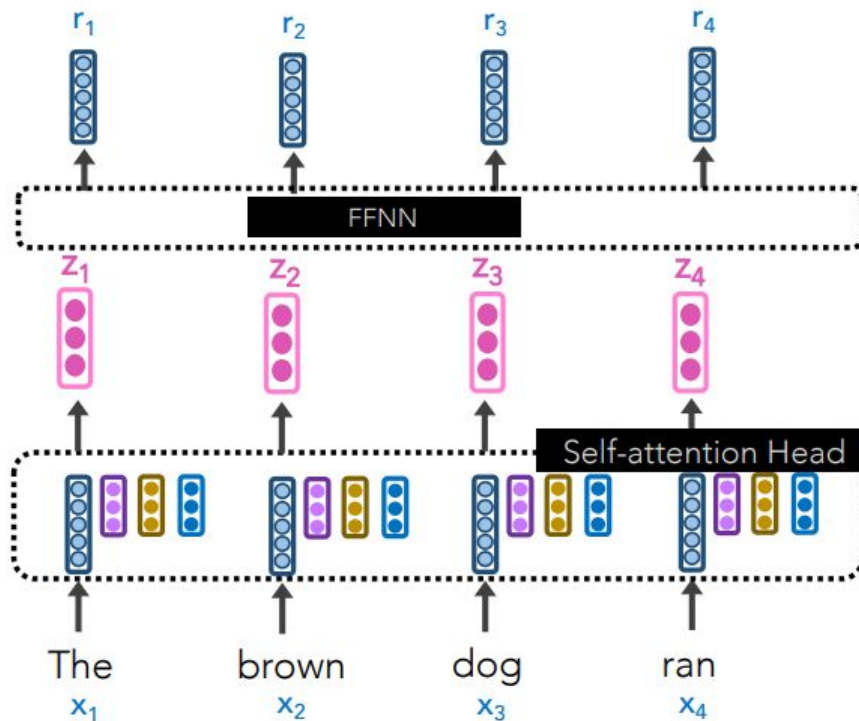


Self-Attention

Tada! Now we have great, new representations z_i via a self-attention head

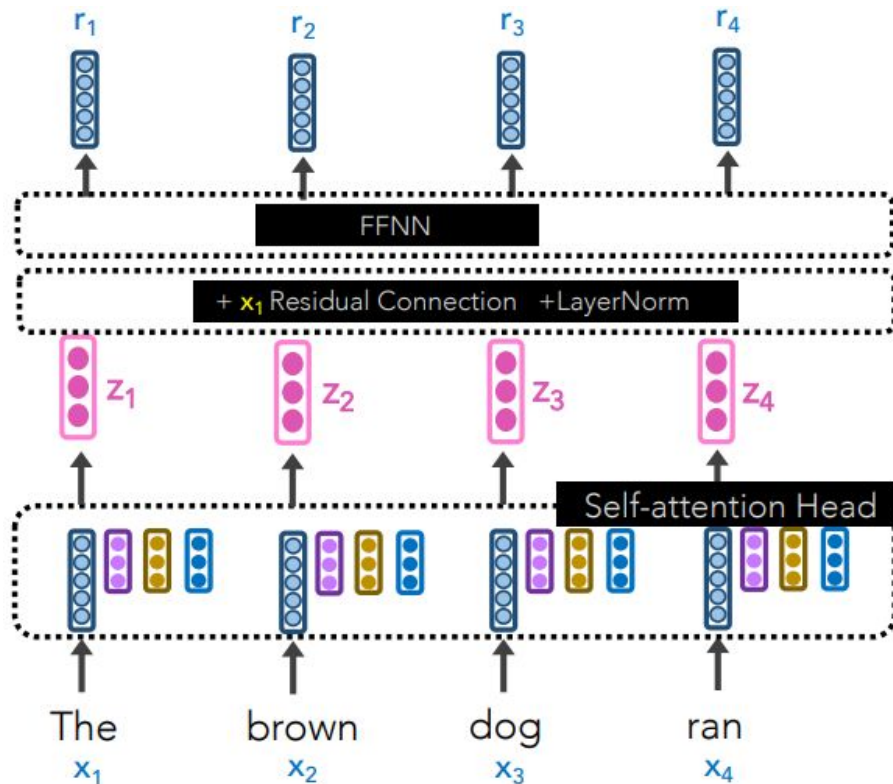


Self Attention + FFNN



Let's further pass each z_i through a FFNN

Self Attention + FFNN

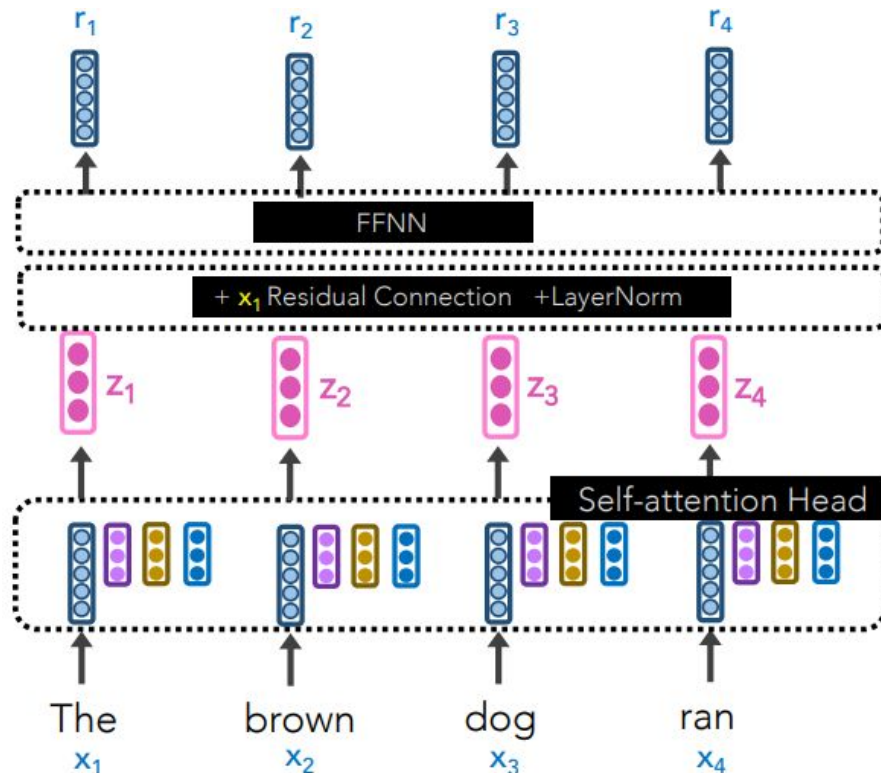


Let's further pass each z_i through a FFNN

We concat w/ a **residual connection** to help ensure relevant info is getting forward passed.

We perform **LayerNorm** to stabilize the network and allow for proper gradient flow.

Self Attention + FFNN



Let's further pass each z_i through a FFNN

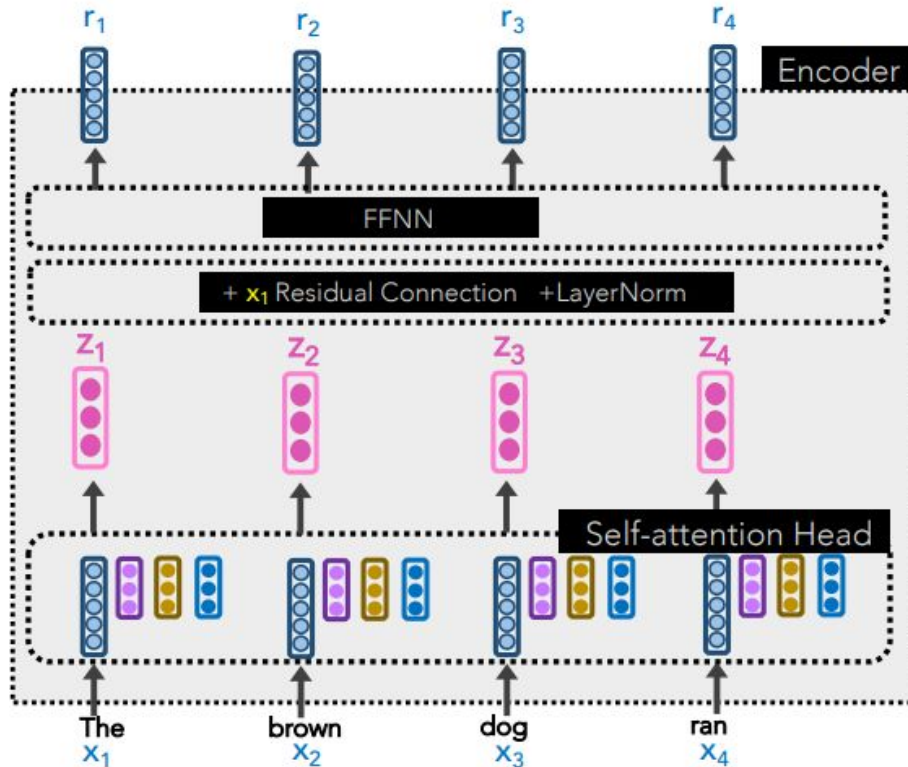
We concat w/ a **residual connection** to help ensure relevant info is getting forward passed.

We perform **LayerNorm** to stabilize the network and allow for proper gradient flow.

Each z_i can be computed in **parallel**, unlike LSTMs!

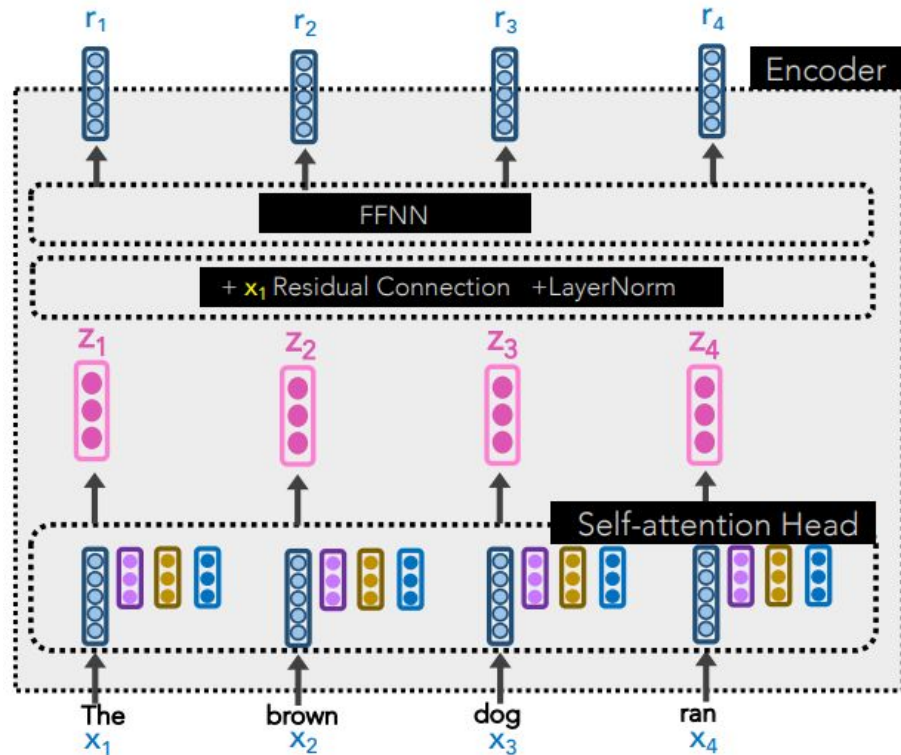
Transformer Encoder

Transformer Encoder



Yay! Our r_i vectors are our new representations, and this entire process is called a **Transformer Encoder**

Transformer Encoder - Positional Encoding

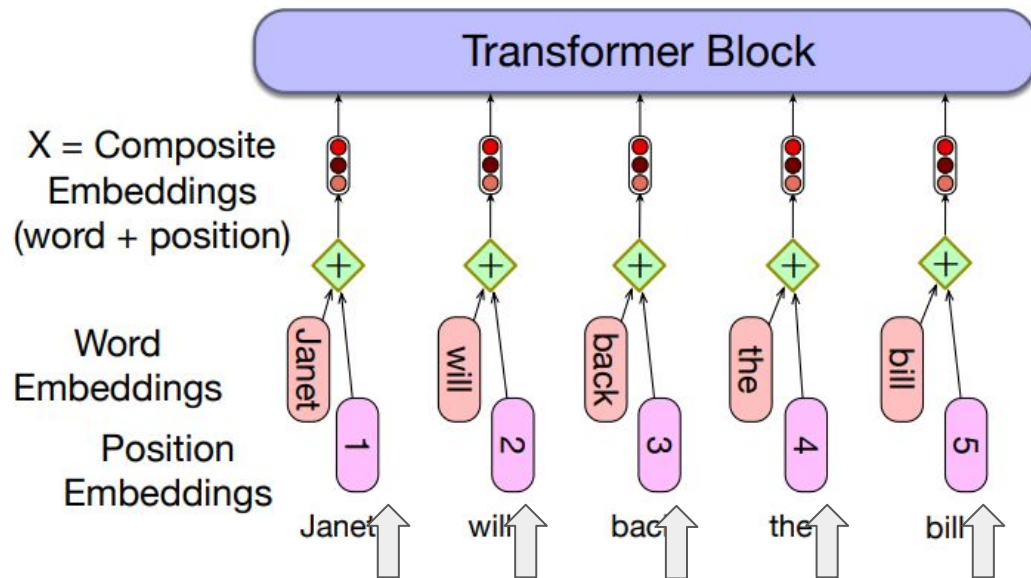


Yay! Our r_i vectors are our new representations, and this entire process is called a **Transformer Encoder**

Problem: there is no concept of positionality. Words are weighted as if a "bag of words"

Solution: append each input word x_i with a **positional encoding**: $\sin(i) \cos(i)$

Transformer Encoder - Positional Encoding



Yay! Our r_i vectors are our new representations, and this entire process is called a **Transformer Encoder**

Problem: there is no concept of positionality. Words are weighted as if a "bag of words"

Solution: append each input word x_i with a **positional encoding**: $\sin(i) \cos(i)$

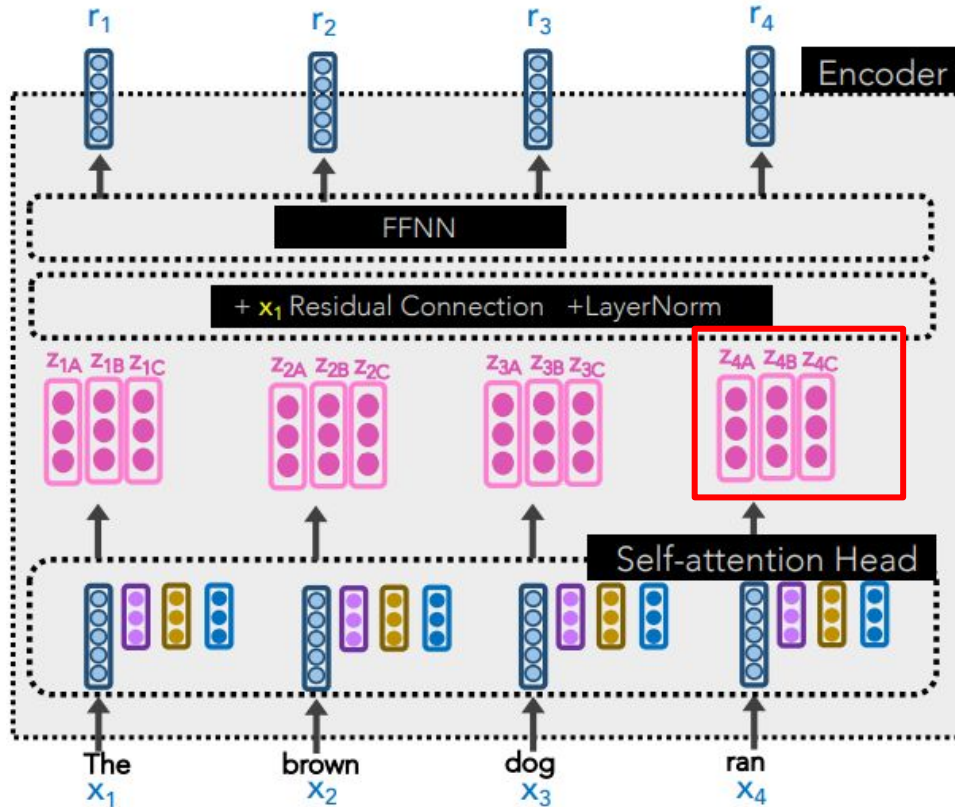
A **Self-Attention Head** has just one set of
query/key/value weight matrices w_q, w_k, w_v

Words can relate in many ways, so it's restrictive to
rely on just one Self-Attention Head in the system.

Let's create Multi-headed Self-Attention

Multi-Head Self Attention

Transformer Encoder

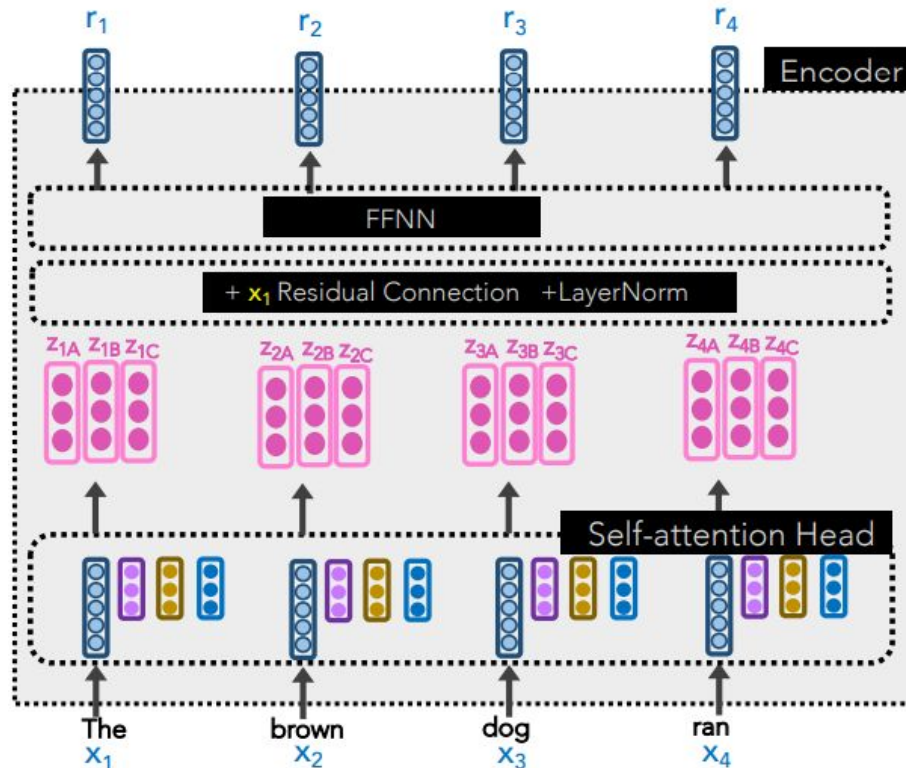


Each **Self-Attention Head** produces a z_i vector.

We can, in parallel, use **multiple heads** and concat the z_i 's.

Multi-Head Self Attention

Transformer Encoder

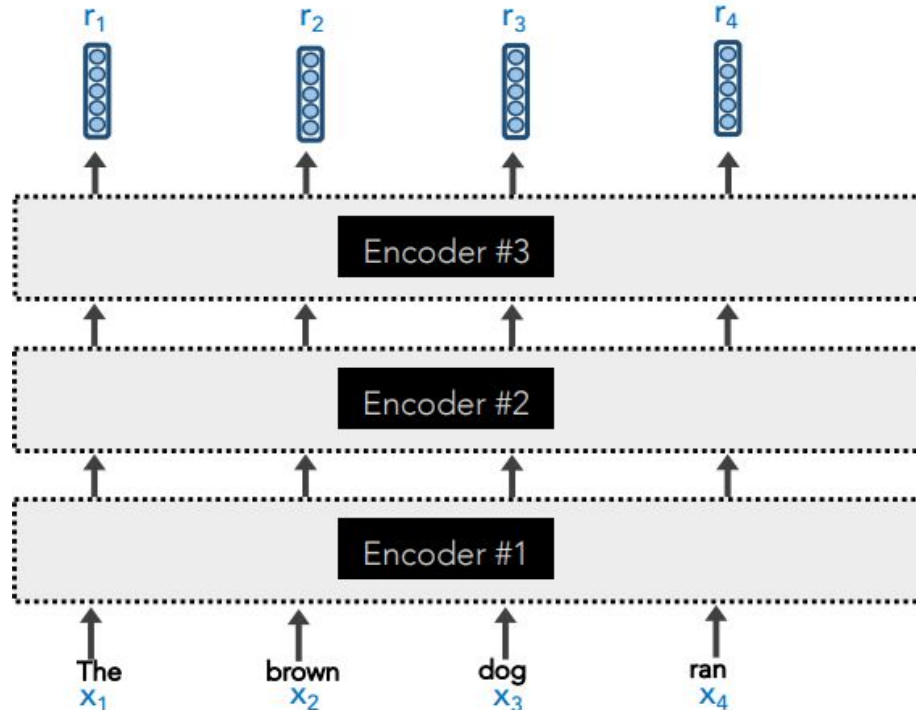


Each **Self-Attention Head** produces a z_i vector.

We can, in parallel, use **multiple heads** and concat the z_i 's.

What about Several Encoders Stacking!!

Transformer Encoder



To recap: all of this looks fancy, but ultimately it's just producing a very good **contextualized embedding** r_i of each word x_i

Why stop with just 1 **Transformer Encoder**?
We could stack several!

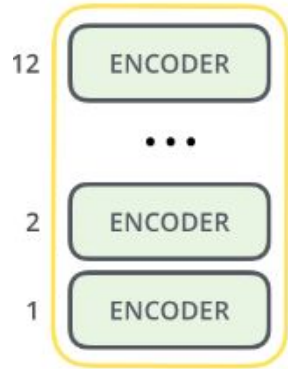
What about Several Encoders Stacking!! - Called **BERT**



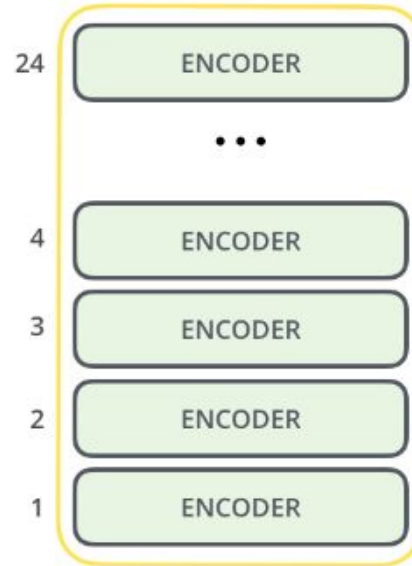
Quest
Intelligence



Also Called as Encoders Only Model



BERT_{BASE}



BERT_{LARGE}

To recap: all of this looks fancy, but ultimately it's just producing a very good **contextualized embedding** r_i of each word x_i

Why stop with just 1 **Transformer Encoder**?
We could stack several!

BERT (a Transformer variant)

BERT is trained on a lot of text data:

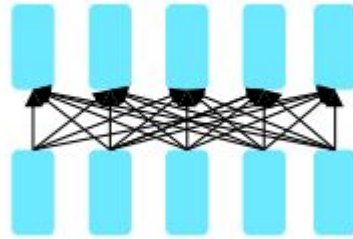
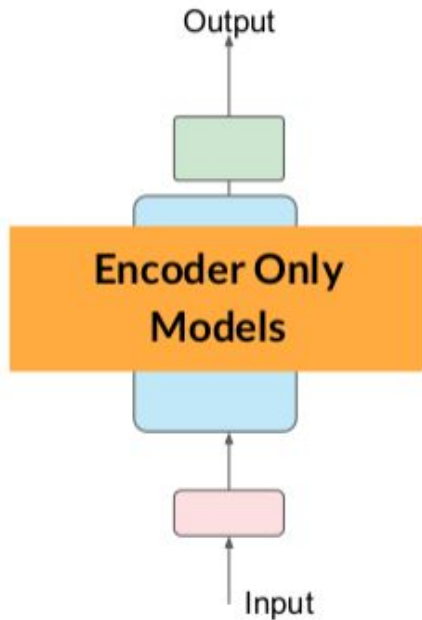
- BooksCorpus (800M words)
- English Wikipedia (2.5B words)

Yay, for transfer learning!

BERT-Base model has 12 transformer blocks, 12 attention heads,
110M parameters!

BERT-Large model has 24 transformer blocks, 16 attention heads,
340M parameters!

Types of Transformers Models

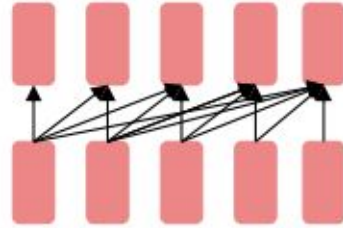
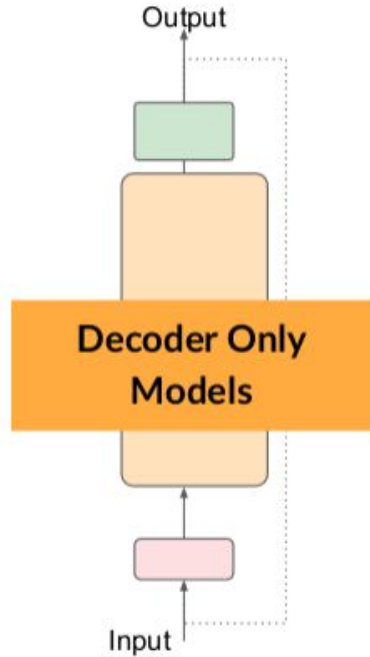


BERT family,
HuBERT

Good use cases:

- Sentiment analysis
- Named entity recognition
- Word classification

Types of Transformers Models

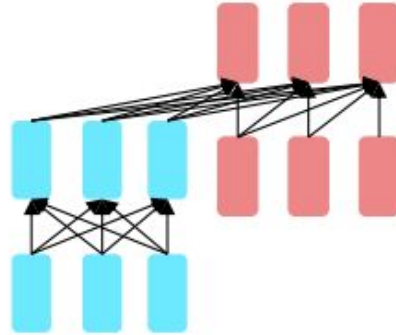
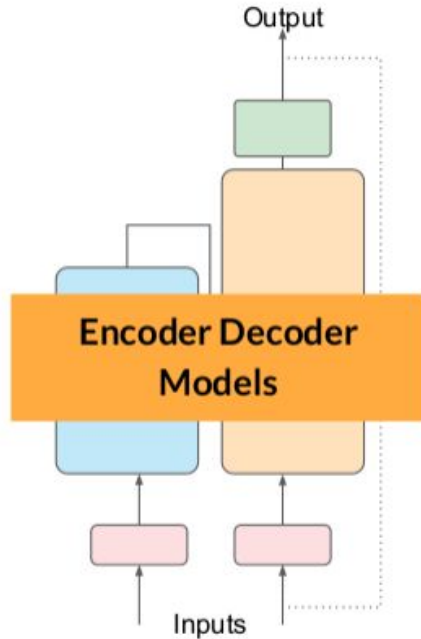


GPT, Claude,
Llama 2
Mixtral

Good use cases:

- Text generation
- Other emergent behavior
 - Depends on model size

Types of Transformers Models



Encoder-decoders
Flan-T5, Whisper

Good use cases:

- Translation
- Text summarization
- Question answering

Transformers Models Size Increasing Over Time

Model size vs. time

